

**REPUBLIC OF CAMEROON
PEACE-WORK-FATHERLAND
UNIVERSITY OF BUEA
BUEA, SOUTH-WEST REGION**



**RÉPUBLIQUE DU CAMEROUN
PAIX-TRAVAIL-PATRIE
UNIVERSITÉ DE BUEA
BUEA, RÉGION DU SUD-OUEST**

**FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING, SE
CEF440: INTERNET PROGRAMMING**

**TASK 6: Database Design and
Implementation**

Report by: GROUP 11

Department: Computer Engineering

CHECKAR – SMART CAR DIAGNOSTICS APPLICATION

**Course Facilitator:
Dr. NKEMENI VALERY**

**Date Issued:
14TH JUNE 2026**

Academic Year: 2025/2026

Name	Matricule
FUNWI KELSEA NDOHNWI	FE23A066
NGADANG ASSONFACK P. ESTELLE	FE23A109
ATEH SWIRRI FOFANG	FE23A018
TABOT GHISLAIN OROCK	FE23A146

Executive Summary

This report presents the Database Design and Implementation phase of the CheckAR Smart Car Diagnostics Application. The database serves as the backbone of the system, enabling secure storage, retrieval, and management of user information, diagnostic reports, maintenance records, mechanic information, and machine learning prediction results.

The report covers the complete database development lifecycle, beginning with data element identification and conceptual database design. It further discusses the Entity Relationship (ER) model, database implementation using Django, PostgreSQL, backend integration using Python and FastAPI, and database connectivity strategies.

The proposed architecture emphasizes scalability, security, maintainability, and efficient data management to support real-time vehicle diagnostics and maintenance tracking functionalities.

Table of Contents

	Page
Executive Summary.....	3
Table of Contents.....	4
1. Introduction.....	6
1.1 Project Overview.....	6
1.2 Document Purpose.....	6
1.3 Scope and Objectives.....	6
1.4 Technical Requirements.....	6
2. Data Elements Analysis.....	7
2.1 Core Data Elements.....	7
2.2 System Data Elements.....	9
3. Conceptual Design.....	10
3.1 Database Architecture.....	10
3.2 Design Principles.....	10
3.3 Data Relationships.....	12
4. Entity Relationship Analysis.....	14
4.1 ER Diagram Structure.....	14
4.2 Relationship Analysis.....	16
4.3 Structural Patterns.....	17
5. Database Implementation.....	18
5.1 PostgreSQL Database Implementation.....	18
5.2 Database Security.....	19
6. Backend Implementation Strategy.....	21
6.1 Technology Stack.....	21
6.2 Database Connection Management.....	22
6.3 API Design Patterns.....	23
6.4 Backend Service Modules.....	23
6.5 Development and Deployment.....	25
7. Integration and Connectivity.....	26
7.1 Authentication Integration.....	27
7.2 Diagnostic Integration.....	28
7.3 Machine Learning Integration.....	29
7.4 Overall System Connectivity.....	30
8. Conclusion.....	31

	Page
8.1 Implementation Summary.....	31
8.2 Key Achievements.....	31
8.3 Future Enhancements.....	31
8.4 Recommendations.....	31

1. Introduction

1.1 Project Overview

CheckAR is an AI-powered mobile application designed to assist vehicle owners in diagnosing car faults through dashboard warning light recognition and engine sound analysis. The application uses machine learning models and cloud-based services to provide diagnostic recommendations and maintenance assistance.

The database plays a critical role by storing diagnostic histories, user profiles, vehicle information, maintenance schedules, and prediction results.

1.2 Document Purpose

The purpose of this document is to:

- Define the database structure of the CheckAR system.
- Identify and analyze system data elements.
- Present the conceptual and logical database designs.
- Explain database implementation using PostgreSQL
- Describe backend integration with the database.
- Demonstrate connectivity between application services and database resources.

1.3 Scope and Objectives

The objectives of the database design are:

- Provide secure storage for all application data.
- Support efficient retrieval of diagnostic information.
- Ensure data integrity and consistency.
- Facilitate backend communication with frontend services.
- Support future scalability and additional system features.

1.4 Technical Requirements

The database solution must satisfy the following requirements:

- Cloud-hosted architecture
- Relational database management system
- Secure authentication mechanisms
- Real-time data synchronization
- Backup and recovery support
- RESTful API integration
- Scalability for increasing user demand

2. Data Elements Analysis

2.1 Core Data Elements

2.1.1 User Management

The user management subsystem encompasses comprehensive user identity and access control mechanisms:

- **User ID:** Primary identifier utilizing UUID format for global uniqueness
- **Email:** User authentication credential with unique constraint enforcement
- **Password Hash:** Cryptographically secure password storage using bcrypt algorithm
 - **Role:** User classification supporting role-based access control (car_user, administrator)
- **Profile Information:** Extended user attributes including name, phone, and preferences
- **Registration Date:** Account creation timestamp for audit and analytics purposes
- **Last Login:** User activity tracking for security monitoring
- **Session Token:** Authentication token management with expiration controls
- **Account Status:** User account state management (active, inactive, suspended)

2.1.2 Diagnostic Records

The diagnostic records system captures comprehensive fault detection sessions:

- **Diagnostic ID:** Unique session identifier enabling precise diagnostic tracking
- **User ID:** Foreign key establishing user-diagnostic relationship
- **Diagnostic Type:** Classification of diagnostic method (dashboard_scan, engine_sound, combined)
- **Timestamp:** Precise diagnostic execution time for temporal analysis
- **Image Data:** Dashboard photograph storage path for visual analysis
- **Audio Data:** Engine sound recording path for acoustic analysis
- **Analysis Results:** Machine learning model output in structured JSON format
- **Confidence Score:** Diagnostic accuracy percentage for reliability assessment
- **Urgency Level:** Risk categorization (immediate, soon, monitoring)
- **Status:** Diagnostic session state (completed, in_progress, failed)

2.1.3 Warning Lights Catalog

The warning lights catalog provides standardized dashboard indicator reference:

- **Light ID:** Unique identifier for each warning light type
- **Symbol Name:** Standardized nomenclature for warning indicators
- **Icon Path:** File system path to visual representation

- **Manufacturer:** Vehicle manufacturer specificity (universal, brand-specific)
- **Description:** Comprehensive explanation of warning significance
- **Severity:** Risk level classification (critical, moderate, informational)
- **Common Causes:** Array of potential underlying issues
- **Repair Suggestions:** Recommended remedial actions

2.1.4 Engine Sound Patterns

The engine sound pattern catalog enables acoustic fault identification:

- **Sound ID:** Unique identifier for distinct sound patterns
- **Sound Type:** Categorical classification (knocking, squealing, hissing, grinding)
- **Description:** Detailed acoustic characteristic description
- **Frequency Range:** Audio frequency spectrum characteristics
- **Severity Level:** Risk assessment (critical, moderate, normal)
- **Common Causes:** Potential mechanical failure modes
- **Repair Suggestions:** Recommended maintenance procedures

2.1.5 Maintenance Records

The maintenance tracking system manages service scheduling and history:

- **Maintenance ID:** Unique maintenance session identifier
- **User ID:** Foreign key linking maintenance to user account
- **Diagnostic ID:** Associated diagnostic session reference
- **Maintenance Type:** Service classification (scheduled, corrective, preventive)
- **Scheduled Date:** Planned maintenance execution timestamp
- **Completed Date:** Actual service completion timestamp
- **Reminder Status:** Notification state (pending, sent, completed)
- **Notes:** Additional maintenance documentation

2.1.6 Mechanic Directory

The mechanic directory facilitates service provider discovery and engagement:

- **Mechanic ID:** Unique service provider identifier
- **Business Name:** Professional service establishment name
- **Contact Information:** Communication channels (phone, email, website)
- **Address:** Physical service location
- **Coordinates:** Geographic positioning (latitude, longitude)
- **Services Offered:** Array of available service categories
- **Rating:** User-generated quality assessment
- **Working Hours:** Service availability schedule
- **Certification:** Professional qualification documentation

2.1.7 Tutorial Content

The tutorial system provides educational resources for diagnostic understanding:

- **Tutorial ID:** Unique educational content identifier
- **Title:** Descriptive tutorial title
- **Description:** Comprehensive content summary
- **YouTube URL:** External video resource link
- **Related Issues:** Array of applicable diagnostic scenarios
- **Duration:** Video content length specification
- **Difficulty Level:** Complexity classification (beginner, intermediate, advanced)
- **View Count:** Usage analytics for content optimization
- **Last Updated:** Content freshness timestamp

2.2 System Data Elements

2.2.1 Application Settings

System configuration management enables dynamic application behavior control:

- **Setting Name:** Configuration parameter designation
- **Setting ID:** Unique configuration parameter identifier
- **Setting Value:** Current parameter value
- **Data Type:** Value type specification (string, integer, boolean, json)
- **Description:** Configuration parameter documentation
- **Category:** Logical grouping (authentication, ml_models, notifications)

2.2.2 ML Model Metadata

Machine learning model management supports AI diagnostic capabilities:

- **Model ID:** Unique artificial intelligence model identifier
- **Model Type:** Algorithm classification (image_classification, audio_classification)
- **Version:** Model iteration number for version control
- **File Path:** Model binary storage location
- **Accuracy:** Performance metrics for model assessment
- **Training Date:** Model development timestamp
- **Status:** Model lifecycle state (active, deprecated, testing)
- **Size:** Model file size for resource management

2.2.3 Audit Logs

Comprehensive audit logging ensures security compliance and system monitoring:

- **Log ID:** Unique audit event identifier
- **User ID:** Actor identification for action attribution
- **Action Type:** Event classification (login, diagnostic, admin_action)
- **Timestamp:** Precise event occurrence time
- **Details:** Structured event details in JSON format
- **IP Address:** Network source identification
- **Status:** Event outcome classification (success, failure, warning)

3. Conceptual Design

3.1 Database Architecture

The CheckAR database follows a relational architecture implemented using PostgreSQL

The architecture consists of:

Presentation Layer

- Flutter Mobile Application

Application Layer

- Fast API Backend Services
- Authentication Services
- Machine Learning Services

Data Layer

- PostgreSQL Database
- File Storage Buckets
- Authentication Services

This layered architecture improves modularity and maintainability.

3.2 Design Principles

3.2.1 Data Integrity

Primary Key Constraints:

All tables implement UUID-based primary keys ensuring global uniqueness and supporting distributed architecture requirements. UUID generation utilizes cryptographically secure random number generation for collision resistance.

Foreign Key Relationships:

Comprehensive referential integrity enforcement through foreign key constraints with appropriate cascade behaviors. Delete cascades are carefully configured to prevent orphaned records while preserving audit trail integrity.

Check Constraints:

Extensive data validation through check constraints ensuring data quality at the database level. Enumerated values are enforced through CHECK constraints rather than separate lookup tables for performance optimization.

Unique Constraints:

Critical business rules are enforced through unique constraints, particularly for email addresses, session tokens, and other naturally unique identifiers.

3.2.2 Security Implementation

Encrypted Storage:

Sensitive data fields utilize column-level encryption for protection at rest. Password hashing implements bcrypt with configurable work factors to resist brute-force attacks while maintaining authentication performance.

Session Token Management:

Secure session management through cryptographically random token generation with configurable expiration policies. Token rotation capabilities support enhanced security for long-lived sessions.

Role-Based Access Control:

Comprehensive RBAC implementation with hierarchical role structures supporting fine-grained permission management. Role assignments are auditable and support temporal access controls.

Row Level Security: PostgreSQL RLS policies ensure users can only access their own data, providing defense-in-depth security even in the event of application-level vulnerabilities.

3.2.3 Performance Optimization

Indexing Strategy:

Strategic B-tree and GIN index deployment on frequently queried columns. Composite indexes support complex query patterns while minimizing index maintenance overhead.

Query Optimization:

Database schema design optimized for common query patterns identified through application requirements analysis. Denormalization is selectively applied where read performance benefits justify the trade-offs.

Data Type Selection:

Optimal data type selection balancing storage efficiency with query performance. JSONB is utilized for semi-structured data requiring query capabilities while maintaining flexibility.

Caching Strategy:

Multi-level caching approach incorporating database query result caching, application-level object caching, and CDN-based static asset caching for comprehensive performance optimization.

3.2.4 Scalability Architecture

Horizontal Scaling:

Database architecture supports horizontal scaling through partitioning strategies for large tables. Time-based partitioning is implemented for diagnostic records and audit logs to manage growth effectively.

Partition Strategy:

Range partitioning based on temporal dimensions for time-series data. Hash partitioning for user-related data to distribute load evenly across multiple database nodes. **Archive Strategy:** Automated data lifecycle management with configurable retention policies. Historical data archival maintains compliance requirements while optimizing active database performance.

Connection Pooling:

Enterprise-grade connection pooling supporting high concurrent user loads while managing database resource utilization efficiently.

3.3 Data Relationships

3.3.1 One-to-Many Relationships

User → Diagnostic Records:

Each user can initiate multiple diagnostic sessions throughout their application usage. This relationship supports comprehensive diagnostic history tracking and enables personalized recommendations based on historical patterns.

User → Maintenance Records:

Users can schedule and track multiple maintenance activities over time. This relationship enables proactive maintenance reminding and comprehensive service history management.

Warning Light → Diagnostic Results:

Individual warning lights can be detected across multiple diagnostic sessions, enabling pattern recognition and trend analysis for improved diagnostic accuracy.

Engine Sound → Diagnostic Results:

Engine sound patterns may be identified in multiple diagnostic contexts, supporting acoustic signature learning and improved classification accuracy over time.

3.3.2 Many-to-Many Relationships

Tutorials ↔ Warning Lights:

Educational content can address multiple warning light scenarios, while individual warning lights may be explained by multiple tutorial resources. Junction table implementation enables flexible content association.

Tutorials ↔ Engine Sounds:

Tutorial content can cover multiple engine sound patterns, supporting comprehensive educational resource organization and discovery.

Mechanics ↔ Services:

Service providers can offer multiple service categories, while individual services may be provided by multiple mechanics. This relationship supports flexible service discovery and comparison.

3.3.3 One-to-One Relationships

User → User Profile:

Each user account has exactly one associated profile containing extended personal information. This separation supports privacy controls and optional profile completion.

Diagnostic → Diagnostic Results:

Each diagnostic session generates exactly one comprehensive result set, ensuring clear result attribution and simplified data model understanding.

4. Entity Relationship Analysis

4.1 ER Diagram Structure

The ER diagram consists of the following entities:

- User
- Vehicle
- Diagnostic Report
- Maintenance Record
- Mechanic
- Notification
- Authentication Session
- ML Model Log

4.1.1 Entity Definitions

The Entity-Relationship diagram for CheckAR AI encompasses 12 core entities, each representing critical components of the automotive diagnostic system.

4.1.2 Primary Entities

Users Entity:

Serves as the central hub for user account management, storing essential authentication credentials and account status information. The entity maintains referential integrity with related diagnostic and maintenance records.

User_Profiles Entity:

Provides extended user information in a one-to-one relationship with Users, supporting optional profile completion and privacy controls through data separation.

Diagnostic_Records Entity:

Core operational entity logging each diagnostic session with comprehensive metadata including diagnostic type, timestamps, and associated multimedia data paths.

Diagnostic_Results Entity:

Stores analyzed outcomes from diagnostic sessions, including confidence scores, urgency levels, and structured analysis results from machine learning models.

Warning_Lights Entity:

Comprehensive catalog of dashboard warning indicators with standardized interpretations, severity classifications, and repair recommendations.

Engine_Sounds Entity:

Acoustic fault pattern catalog enabling sound-based diagnosis with frequency characteristics and associated mechanical failure modes.

Maintenance_Records Entity:

Tracks scheduled and completed vehicle service activities with comprehensive scheduling and reminder management capabilities.

Mechanics Entity:

Directory of registered service providers with location data, contact information, ratings, and service capability documentation.

Services Entity:

Catalog of available maintenance and repair services with cost estimates and categorical organization.

Tutorials Entity:

Educational content management supporting diagnostic understanding with difficulty levels and usage analytics.

ML_Models Entity:

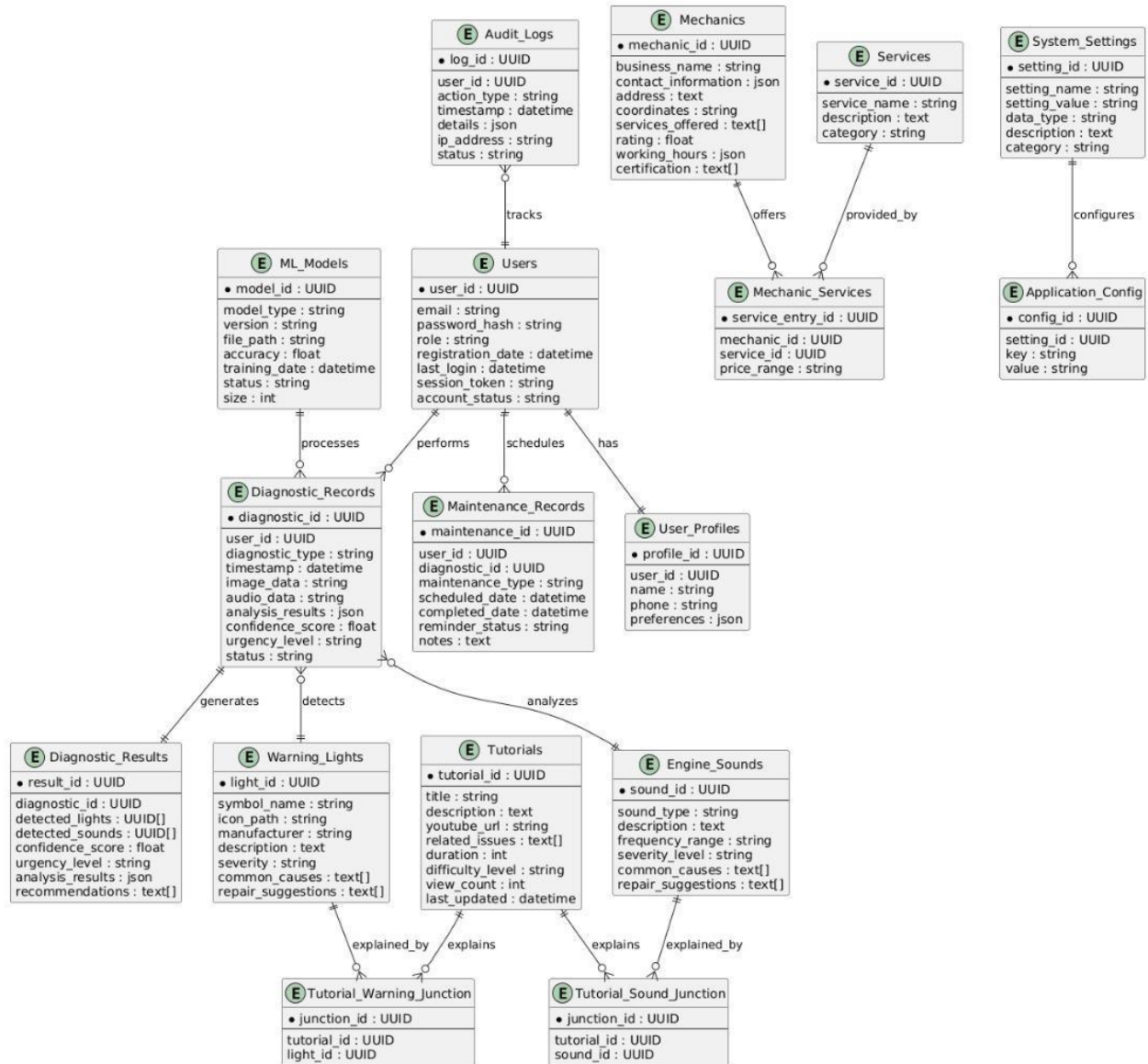
Machine learning model metadata management supporting version control, performance tracking, and deployment management.

Audit_Logs Entity:

Comprehensive system activity tracking for security monitoring and regulatory compliance.

The diagram represents entity attributes, primary keys, foreign keys, and relationships.

ER Diagram



(Figure 4.1: CheckAR Entity Relationship Diagram)

4.2 Relationship Analysis

4.2.1 One-to-Many Relationships

The diagram implements several critical one-to-many relationships that form the backbone of the system's data architecture:

Parent Entity	Child Entity	Relationship Description
Users	Diagnostic_Records	Users can initiate multiple diagnostic sessions
Users	Maintenance_Records	Users can schedule multiple maintenance tasks
Diagnostic_Records	Diagnostic_Results	Each diagnostic generates multiple result components
ML_Models	Diagnostic_Records	Models process multiple diagnostic sessions

4.2.2 Many-to-Many Relationships

Complex many-to-many relationships are implemented through junction tables to support flexible content association:

Entity 1	Entity 2	Junction Table	Purpose
Tutorials	Warning_Lights	Tutorial_Warning_Junction	cont indicators
Tutorials	Engine_Sounds	Tutorial_Sound_Junction	Links tutorials to engine sound diagnoses
Mechanics	Services	Mechanic_Services	ce pr offerings

4.2.3 One-to-One Relationships

Strategic one-to-one relationships optimize data organization and privacy controls:

Primary Entity	Related Entity	Relationship Purpose
Users	User_Profiles	Separates core authentication from extended profile data
Diagnostic_Records	Diagnostic_Results	Ensures single result set per diagnostic session

4.3 Structural Patterns

4.3.1 Central Hub Architecture

The Users entity serves as the primary architectural hub, connecting to multiple operational entities:

- Diagnostic operations through Diagnostic_Records
- Maintenance management through Maintenance_Records

- Security tracking through Audit_Logs
- Profile management through User_Profiles

4.3.2 Diagnostic Workflow Pattern

The diagnostic workflow follows a structured pattern from initiation to result delivery:

1. User initiates diagnostic session (Diagnostic_Records)
2. ML_Models process submitted data
3. Results are stored (Diagnostic_Results)
4. Warning_Lights and Engine_Sounds provide interpretation
5. Tutorials offer educational resources
6. Maintenance_Records track follow-up actions

5. Database Implementation

5.1 PostgreSQL Database Implementation

The CheckAR Smart Car Diagnostics Application utilizes PostgreSQL as its primary Relational Database Management System (RDBMS). PostgreSQL was selected due to its high performance, reliability, scalability, and seamless integration with the Django framework. As an open-source enterprise-grade database solution, PostgreSQL provides advanced data management capabilities that are suitable for handling the large volumes of information generated by the application.

The database serves as the central repository for all system data, including user accounts, vehicle profiles, diagnostic reports, maintenance records, mechanic information, notifications, and machine learning prediction results. By organizing data into structured relational tables, PostgreSQL enables efficient storage, retrieval, and management of information while maintaining data consistency and integrity across the entire system.

The implementation of PostgreSQL supports the operational requirements of CheckAR by ensuring that users can securely access their diagnostic history, track vehicle maintenance schedules, and receive intelligent recommendations generated by the machine learning models. Furthermore, PostgreSQL's support for concurrent transactions allows multiple users to interact with the system simultaneously without compromising performance or data accuracy.

Key Features of PostgreSQL Used

Relational Database Architecture

PostgreSQL employs a relational database structure in which data is organized into tables connected through defined relationships. This architecture minimizes redundancy, improves consistency, and simplifies data management. Relationships between entities such as users, vehicles, diagnostic reports, and maintenance records are established through primary and foreign keys.

ACID-Compliant Transactions

PostgreSQL supports Atomicity, Consistency, Isolation, and Durability (ACID) properties, ensuring that all database transactions are executed reliably. This feature is particularly important when storing diagnostic results and user information, as it guarantees that incomplete or failed transactions do not corrupt the database.

Foreign Key and Integrity Constraints

Data integrity is maintained through the use of foreign key constraints, unique constraints, and validation rules. These mechanisms ensure that relationships between tables remain valid and that only accurate and consistent data can be stored within the database.

Indexing for Faster Query Execution

Indexes are created on frequently accessed columns such as user identifiers, vehicle identifiers, and diagnostic records. Indexing significantly improves query performance, enabling users to retrieve diagnostic histories and maintenance records quickly even as the database grows.

Role-Based Security Management

PostgreSQL provides robust access control mechanisms that restrict database operations based on user roles and permissions. Administrative users are granted elevated privileges, while standard users are limited to accessing only their own data.

Backup and Recovery Support

Regular backup procedures are implemented to prevent data loss caused by hardware failures, software errors, or unexpected system interruptions. PostgreSQL's backup and recovery tools enable the restoration of critical data and ensure business continuity.

Scalability for Future Expansion

The database architecture has been designed with scalability in mind. PostgreSQL supports vertical and horizontal scaling techniques, allowing the system to accommodate an increasing number of users, vehicles, and diagnostic records as the application evolves.

5.2 Database Security

Database security is a critical component of the CheckAR system because it handles sensitive user information and diagnostic data. Several security measures have been implemented to protect the confidentiality, integrity, and availability of stored information.

User Authentication and Authorization

Authentication is managed through Django's built-in authentication framework, which verifies user identities before granting access to system resources. Authorization mechanisms ensure that users can only perform actions permitted by their assigned roles.

Password Hashing

User passwords are never stored in plain text. Django automatically hashes passwords using secure cryptographic algorithms before storing them in the database. This approach significantly reduces the risk of credential theft and unauthorized access.

Role-Based Access Control (RBAC)

Role-Based Access Control is used to define different levels of access within the application. Standard users can manage their own vehicle and diagnostic information, while administrators have additional privileges to manage users, system settings, and machine learning models.

Secure Storage of Database Credentials

Sensitive database credentials are stored in environment variables rather than hardcoded within the application source code. This practice minimizes security risks and facilitates secure deployment across different environments.

Protection Against SQL Injection

The Django Object Relational Mapper (ORM) is used for database interactions. By automatically parameterizing queries, the ORM protects the application against SQL injection attacks, one of the most common database security vulnerabilities.

Database Backup and Recovery Procedures

Automated backup mechanisms are implemented to create regular copies of the database. These backups can be used to restore the system in the event of accidental data deletion, system failure, or cybersecurity incidents.

Overall, the PostgreSQL implementation provides a secure, scalable, and efficient data management solution that supports the functional and non-functional requirements of the CheckAR Smart Car Diagnostics Application.

6. Backend Implementation Strategy

6.1 Technology Stack

The backend of the CheckAR Smart Car Diagnostics Application is built using a modern and scalable technology stack designed to support secure data processing, efficient communication between system components, and seamless integration with machine learning services. Each technology was selected based on its performance, reliability, community support, and compatibility with the project requirements.

Technology	Purpose
Python	Backend Programming Language
Django	Web Application Framework
Django REST Framework (DRF)	REST API Development
PostgreSQL	Database Management System
TensorFlow	Machine Learning Model Execution
JWT Authentication	Secure User Authentication
GitHub	Version Control and Collaboration
Docker	Application Containerization

Python

Python serves as the primary backend programming language for the CheckAR application. It was chosen because of its simplicity, extensive library ecosystem, and strong support for artificial intelligence and machine learning development. Python enables rapid development while maintaining code readability and maintainability.

Django

Django is the main web framework used for backend development. It follows the Model-View-Template (MVT) architectural pattern and provides built-in features such as authentication, security protection, database integration, and administrative interfaces. Django significantly reduces development time by offering reusable components and best-practice implementations.

Django REST Framework (DRF)

Django REST Framework extends Django by providing tools for building RESTful APIs. DRF simplifies the creation of API endpoints that allow communication between the mobile application, machine learning services, and the database. It also provides serialization, authentication, permission management, and request validation capabilities.

PostgreSQL

PostgreSQL serves as the relational database management system responsible for storing user information, vehicle records, diagnostic reports, maintenance schedules, mechanic directories, and machine learning outputs. Its reliability and scalability make it suitable for handling large datasets and concurrent user requests.

TensorFlow

TensorFlow is used to execute machine learning models responsible for dashboard warning light recognition and engine sound classification. The framework enables the integration of artificial intelligence capabilities into the diagnostic workflow.

JWT Authentication

JSON Web Tokens (JWT) are used to provide secure user authentication and session management. JWT allows users to remain authenticated while interacting with protected resources without repeatedly providing login credentials.

GitHub

GitHub is used as the version control platform for managing source code, tracking project changes, and facilitating collaboration among team members. It provides a centralized repository where code updates can be reviewed and merged efficiently.

Docker

Docker is utilized for containerization, ensuring that the application runs consistently across different development and deployment environments. Containers package the application together with all dependencies, reducing compatibility issues.

6.2 Database Connection Management

Efficient communication between the backend application and the PostgreSQL database is essential for the successful operation of CheckAR. Django manages database interactions through its Object Relational Mapper (ORM), which acts as an intermediary between Python objects and database tables.

The ORM eliminates the need for writing raw SQL queries for most operations, thereby reducing development complexity and minimizing the risk of SQL injection vulnerabilities. Developers can perform database operations using Python classes and methods, which are automatically translated into optimized SQL commands.

Database credentials such as usernames, passwords, database names, and host information are stored securely using environment variables. This approach prevents sensitive information from being exposed in source code repositories and supports secure deployment practices.

Example Database Configuration

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'checkar_db',
        'USER': 'postgres',
        'PASSWORD': '*****',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

Additionally, connection pooling and query optimization techniques are employed to improve application performance and efficiently manage concurrent database requests.

6.3 API Design Patterns

The CheckAR backend follows a RESTful architectural style implemented using Django REST Framework. REST (Representational State Transfer) enables standardized communication between the frontend application and backend services through HTTP requests.

RESTful APIs provide several benefits including scalability, simplicity, flexibility, and platform independence. Each API endpoint is responsible for performing a specific function and returning structured JSON responses.

Core API Endpoints

Method	Endpoint	Purpose
POST	/api/register	Register New User
POST	/api/login	Authenticate User
POST	/api/diagnose/image	Diagnose Dashboard Warning Lights
POST	/api/diagnose/sound	Diagnose Engine Sounds
GET	/api/history	Retrieve Diagnostic History
GET	/api/mechanics	Retrieve Mechanic Recommendations

When a user submits a dashboard image or engine sound recording, the API forwards the data to the diagnostic service, receives prediction results from the machine learning model, stores the results in the database, and returns a structured response to the frontend application.

6.4 Backend Service Modules

To improve maintainability and modularity, the backend system is divided into specialized service modules, each responsible for handling a specific set of functionalities.

Authentication Module

The Authentication Module manages user identity verification and access control.

Its responsibilities include:

- User registration
- User login and logout
- JWT token generation and validation
- Password reset and management
- Access permission control

This module ensures that only authorized users can access protected resources within the system.

Diagnostic Module

The Diagnostic Module is the core component responsible for AI-powered vehicle diagnosis.

Its responsibilities include:

- Dashboard warning light recognition
- Engine sound analysis
- Machine learning model execution
- Prediction generation
- Diagnostic report creation and storage

The module processes user-submitted diagnostic data and generates intelligent recommendations based on machine learning outputs.

Maintenance Module

The Maintenance Module supports preventive vehicle care by tracking service schedules and maintenance activities.

Its responsibilities include:

- Maintenance scheduling
- Service history management
- Reminder notification generation
- Maintenance record updates

This module helps users maintain their vehicles proactively and reduce the likelihood of future mechanical failures.

Mechanic Module

The Mechanic Module facilitates access to automotive repair professionals.

Its responsibilities include:

- Mechanic directory management
- Workshop recommendations
- Mechanic search functionality
- Contact information retrieval
- Service provider listings

The module enables users to locate qualified mechanics based on diagnostic outcomes and maintenance requirements.

6.5 Development and Deployment

The development and deployment process follows modern software engineering practices to ensure application reliability, maintainability, and scalability.

Version Control

GitHub is used to manage source code versions, track project progress, and coordinate collaborative development activities among team members. Branching and pull-request workflows are employed to maintain code quality.

Containerization

Docker containers package the backend application, dependencies, and runtime environment into a portable unit. This ensures consistent behavior across development, testing, and production environments.

Database Hosting

PostgreSQL is hosted on a dedicated database server, enabling secure and efficient storage of application data. Database backups and monitoring tools are used to maintain availability and reliability.

Continuous Testing and Updates

Automated testing procedures are integrated into the development workflow to detect bugs and performance issues before deployment. Continuous updates ensure that new features and security improvements can be released efficiently.

Cloud Deployment

The application can be deployed on cloud platforms such as Render, Railway, or Amazon Web Services (AWS). Cloud deployment provides scalability, high availability, automated infrastructure management, and support for growing user demand.

Overall, the backend implementation strategy provides a secure, scalable, and maintainable foundation for supporting the intelligent diagnostic capabilities of the CheckAR Smart Car Diagnostics Application.

The backend implementation uses:

Technology	Purpose
Python	Backend Programming
FastAPI	REST API Framework
Supabase	Database Services
PostgreSQL	Relational Database
TensorFlow	AI Model Execution
JWT	Authentication

7. Integration and Connectivity

Integration and connectivity are critical aspects of the CheckAR Smart Car Diagnostics Application because they enable seamless communication between the frontend mobile application, backend services, machine learning models, and the PostgreSQL database. The system follows a service-oriented architecture in which different components interact through well-defined Application Programming Interfaces (APIs).

The integration layer ensures that data submitted by users is securely processed, stored, analyzed, and returned in real time. Through the use of Django REST Framework APIs, the application provides reliable communication channels that support authentication, vehicle diagnostics, maintenance tracking, and machine learning operations.

7.1 Authentication Integration

User authentication is implemented using Django's built-in authentication system combined with JSON Web Tokens (JWT). This integration provides a secure mechanism for verifying user identities and controlling access to protected resources within the application.

The authentication service acts as the first layer of security, ensuring that only authorized users can access vehicle diagnostics, maintenance records, and personal account information. Upon

successful login, the system generates a JWT token that is used for subsequent requests without requiring users to repeatedly enter their credentials.

Authentication Routes

Method	Endpoint	Purpose
POST	/api/register	Register a new user account
POST	/api/login	Authenticate user credentials
POST	/api/logout	End user session
POST	/api/token/refresh	Generate a new access token

Authentication Workflow

The authentication process follows these steps:

1. The user submits registration or login information through the mobile application.
2. Django validates the submitted credentials against records stored in PostgreSQL.
3. If authentication is successful, a JWT access token is generated.
4. The token is returned to the frontend application.
5. The frontend includes the token in subsequent API requests.
6. Protected routes verify the token before granting access to system resources.

Benefits of JWT Authentication

The use of JWT authentication offers several advantages:

- Secure user session management
- Stateless authentication architecture
- Reduced server-side session storage requirements
- Improved scalability
- Enhanced protection against unauthorized access
- Support for mobile and web clients

By implementing Django Authentication and JWT, the CheckAR system ensures that user information remains secure while providing a smooth and efficient user experience.

7.2 Diagnostic Integration

The Diagnostic Integration layer enables communication between the frontend application, backend services, machine learning models, and the PostgreSQL database. This component is responsible for processing vehicle diagnostic requests submitted by users.

Users can upload dashboard warning light images or engine sound recordings through the mobile application. The submitted data is transferred to Django backend services through RESTful APIs, where it is analyzed using machine learning algorithms.

Diagnostic Routes

Method	Endpoint	Purpose
POST	/api/diagnosis/image	Dashboard warning light diagnosis
POST	/api/diagnosis/sound	Engine sound diagnosis
GET	/api/diagnosis/history	Retrieve diagnostic history

Diagnostic Workflow

The diagnostic process consists of the following steps:

1. The user uploads a dashboard image or engine sound recording using the mobile application.
2. The frontend sends the data to the appropriate Django API endpoint.
3. The backend validates the received data and forwards it to the machine learning service.
4. The machine learning model processes the input and generates prediction results.
5. Diagnostic information, confidence scores, and recommendations are stored in PostgreSQL.
6. The backend constructs a response containing the diagnostic results.
7. The results are returned to the mobile application for display to the user.

Database Connectivity During Diagnosis

During the diagnostic process, the backend interacts with the PostgreSQL database to:

- Store uploaded diagnostic records
- Save machine learning prediction results
- Track user diagnostic history
- Record timestamps and confidence scores
- Generate maintenance recommendations

This integration ensures that users can access previous diagnoses and monitor the condition of their vehicles over time.

Benefits of Diagnostic Integration

The integration provides:

- Real-time diagnostic processing
- Automated storage of results

- Fast communication between services
- Improved user experience
- Historical data tracking
- Enhanced system reliability

7.3 Machine Learning Integration

Machine learning integration forms the intelligent core of the CheckAR application. The system incorporates AI models that analyze dashboard warning lights and engine sounds to identify potential vehicle faults.

The machine learning models are integrated into the Django backend through dedicated APIs. This architecture allows the backend to communicate with AI services while maintaining separation between application logic and model processing.

Machine Learning Routes

Method	Endpoint	Purpose
POST	/api/model/predict	Generate diagnostic predictions
GET	/api/model/status	Check model availability and status
POST	/api/model/update	Update or retrain machine learning models

Machine Learning Workflow

The machine learning integration process follows these steps:

1. Diagnostic data is received from the frontend application.
2. Django preprocesses the data into a format suitable for machine learning analysis.
3. The processed data is sent to TensorFlow-based AI models.
4. The models perform classification and prediction operations.
5. Confidence scores and fault classifications are generated.
6. Results are returned to Django services.
7. Prediction data is stored in the database for future reference.
8. The frontend receives diagnostic recommendations and displays them to the user.

Data Storage and Logging

To support auditing, performance evaluation, and historical analysis, machine learning outputs are automatically stored in multiple database tables, including:

- Diagnostic Reports Table

- Machine Learning Model Logs Table
- User Diagnostic History Table

These records enable administrators to monitor model performance, track prediction accuracy, and maintain a complete history of diagnostic activities.

Advantages of Machine Learning Integration

The integration of machine learning services provides several benefits:

- Automated vehicle fault detection
- Faster diagnostic processes
- Improved diagnostic accuracy
- Reduced dependency on manual inspections
- Continuous learning and model improvement
- Intelligent maintenance recommendations

7.4 Overall System Connectivity

The CheckAR system follows a connected architecture in which all major components communicate efficiently through APIs and database services. The mobile application serves as the user interface, while Django acts as the central processing layer coordinating interactions between PostgreSQL and TensorFlow models.

The overall connectivity flow can be summarized as follows:

Mobile Application → Django REST API → Machine Learning Services → PostgreSQL Database → Django REST API → Mobile Application

This architecture ensures secure communication, efficient data management, real-time diagnostics, and scalability for future enhancements. Through proper integration and connectivity mechanisms, CheckAR delivers a reliable and intelligent vehicle diagnostic solution capable of supporting a growing number of users and diagnostic requests.

8. Conclusion

8.1 Implementation Summary

This report presented the complete database design and implementation strategy for the CheckAR Smart Car Diagnostics Application. The design supports secure storage, efficient data retrieval, and seamless integration with AI-powered diagnostic services.

8.2 Key Achievements

Major achievements include:

- Well-structured relational database design
- Complete ER model
- PostgreSQL implementation
- FastAPI backend integration
- Secure authentication architecture
- Scalable cloud-based infrastructure

8.3 Future Enhancements

Future improvements may include:

- Real-time analytics dashboards
- Predictive maintenance algorithms
- Multi-vehicle fleet management
- Advanced reporting modules

8.4 Recommendations

The following recommendations are proposed:

- Implement automated database backups.
- Enable advanced monitoring and logging.
- Introduce caching mechanisms for improved performance.
- Expand database support for future AI services.

The implemented database architecture provides a reliable foundation for the continued development and deployment of the CheckAR Smart Car Diagnostics Application.